PROJECT PROPOSAL

[Memory & Storage Management]

Course Name:	[Operating system]
Prepared by:	[Yehya Nasser Mohame 420230316
	Hana Mohamed Osman 420230690
	Mostafa Mohamed Elsayed 420230023
	Mohaned Ashraf Ahmed 420230298
	Hassan Ramadan Abdelhady 420230136]
Instructor:	[DR/ Mohamed Amer
	ENG/ Ahmed Abdelmo3ty]
Department:	[computer science]
Date:	[3/10/2026]

1. Project Overview (Executive Summary)

This project focuses on optimizing two critical components of an Operating System: Memory Management and Storage Management. Traditional operating systems often rely on reactive algorithms, such as Least Recently Used (LRU) for page replacement, and static file architectures. This project aims to replace these with proactive, intelligent models. We will develop a Predictive Virtual Memory Manager that utilizes user behavior patterns to anticipate required pages, alongside a Self-Organizing Smart File System. The file system will monitor data access frequencies, automatically migrating "hot" (frequently accessed) data to faster storage sectors while compressing and encrypting "cold" (rarely accessed) data. Together, these subsystems aim to significantly enhance overall OS performance and resource utilization

2. Problem Statement & Motivation

Modern applications demand massive memory and fast storage access, yet OS resource management often acts as a bottleneck. Traditional virtual memory managers use reactive algorithms that wait for a page fault to occur before making replacement decisions, leading to latency. Similarly, traditional file systems store data statically and do not adapt to how frequently files are used, resulting in wasted fast-storage space. Our motivation is to bridge the gap between static OS algorithms and dynamic user behavior to create a smarter, self-optimizing operating environment

3. Project Objectives

 Objective 1: Design a Predictive Virtual Memory Manager that anticipates required pages to reduce page faults and improve system fluidity.

 Objective 2: Develop a Self-Organizing Smart File System that categorizes data access frequency into "hot" and "cold" data.

 Objective 3: Implement an automated mechanism to move "hot" data to faster storage and compress/encrypt "cold" data to save space.

4. Methodology & Plan

 Research Phase: We will collect data on user behavior patterns and file access frequencies to build our predictive models.

 Implementation Phase: We will write a C++ application to simulate the memory manager using predictive algorithms instead of LRU, and implement a background daemon running on Ubuntu Linux (via VirtualBox) to reorganize files based on access logs.

 Tools/Software: C/C++ (for core simulation logic), Oracle VM VirtualBox, and Ubuntu Linux (for system testing and background service execution).

5. Expected Outcomes

The final result will be a working simulation (prototype) demonstrated on a Linux environment, showing the predictive memory manager and the smart file system operating together. This will be accompanied by a detailed report comparing its efficiency against traditional OS management techniques.

6. Project Timeline

Phase	Task Description	Deadline
Phase 1	Research & Data Collection	Week 2
Phase 2	Design / Development	Week 5
Phase 3	Testing & Analysis	Week 7
Phase 4	Final Report & Submission	Week 10

7. Resource Requirements

- Development machines (Windows/Mac).
- Oracle VM VirtualBox software.
- Ubuntu Linux ISO image.
- C++ Compiler (GCC/G++).

8. References

https://doi.org/10.1145/3462639

https://doi.org/10.1080/02564602.2022.2100869

https://doi.org/10.1109/HPCA.2019.00053

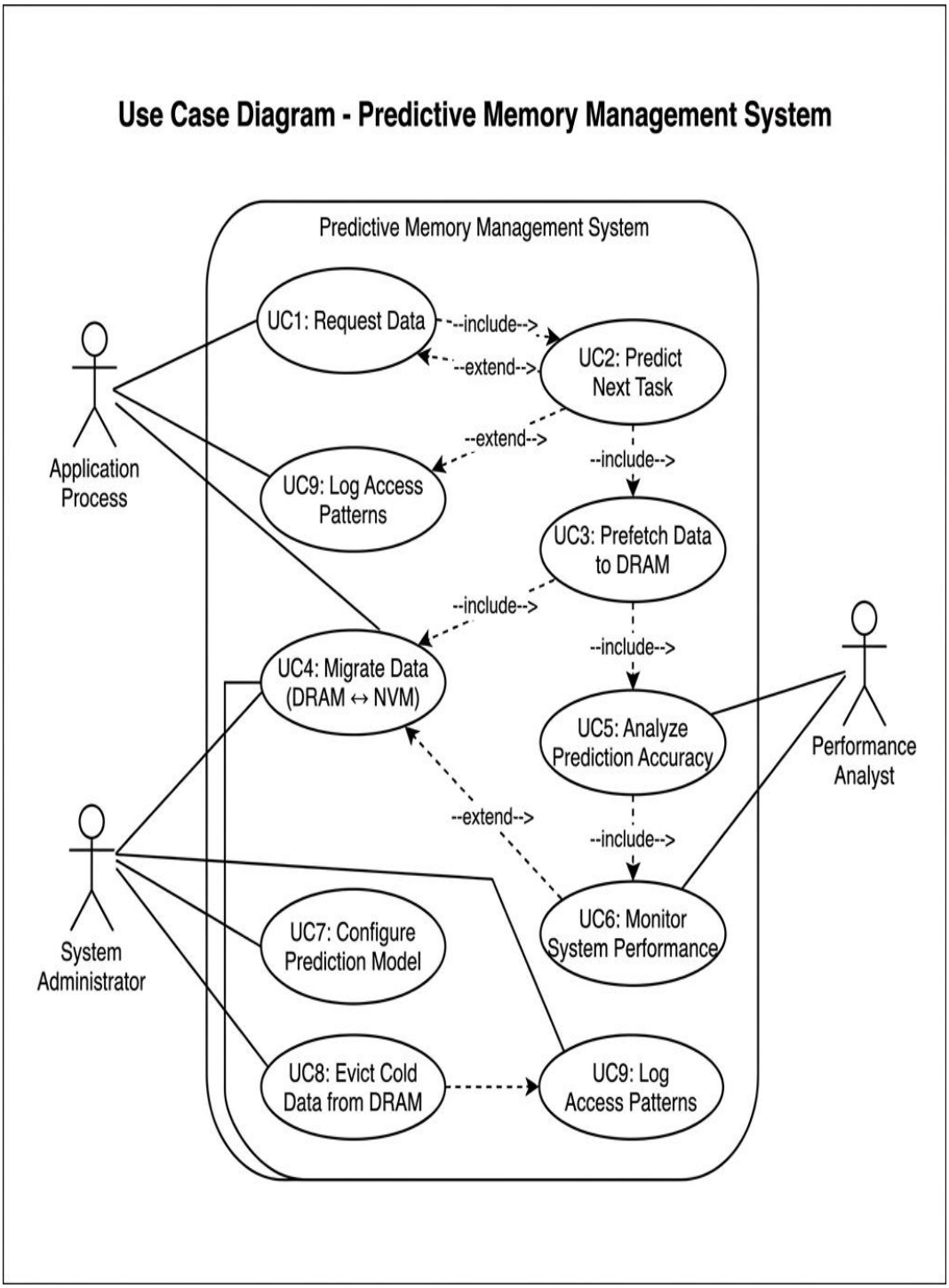
https://doi.org/10.1080/02564602.2022.2100869

https://doi.org/10.1109/HPCA.2019.00053

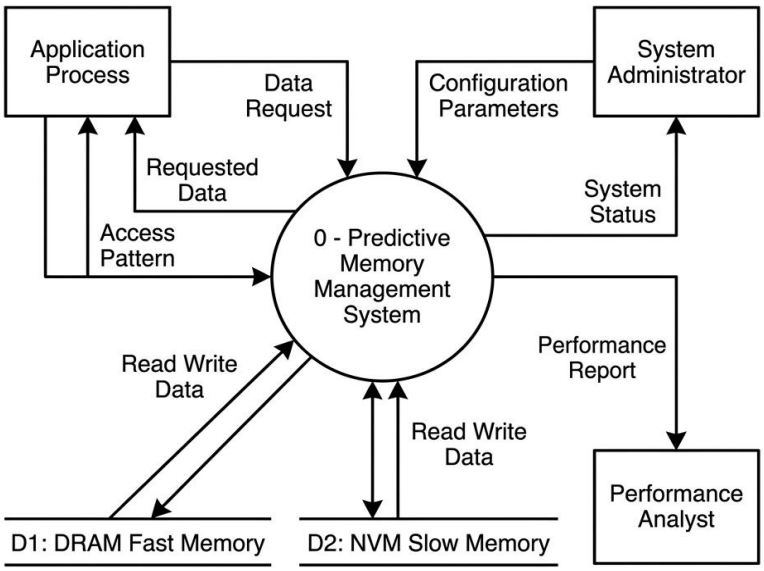
https://doi.org/10.1109/MICRO.2016.7783763

https://doi.org/10.1109/ACCESS.2021.3103524

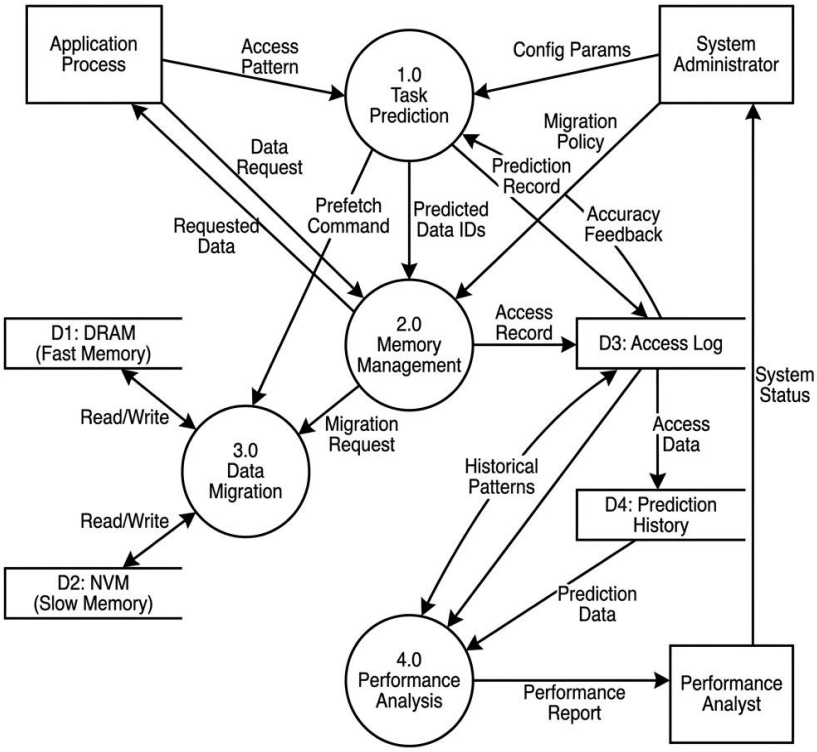
https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=Memory+%26+Storage+Management+&oq=


Use Case Diagram - Predictive Memory Management System
Predictive Memory Management System
UC1: Request Data
--include-->
--extend-->
UC2: Predict Next Task
--extend-->
--include-->
UC9: Log Access Patterns
UC3: Prefetch Data to DRAM
--include-->
--include-->
UC4: Migrate Data (DRAM ↔ NVM)
UC5: Analyze Prediction Accuracy
--extend-->
--include-->
UC6: Monitor System Performance
UC7: Configure Prediction Model
UC8: Evict Cold Data from DRAM
UC9: Log Access Patterns
Application Process
System Administrator
Performance Analyst

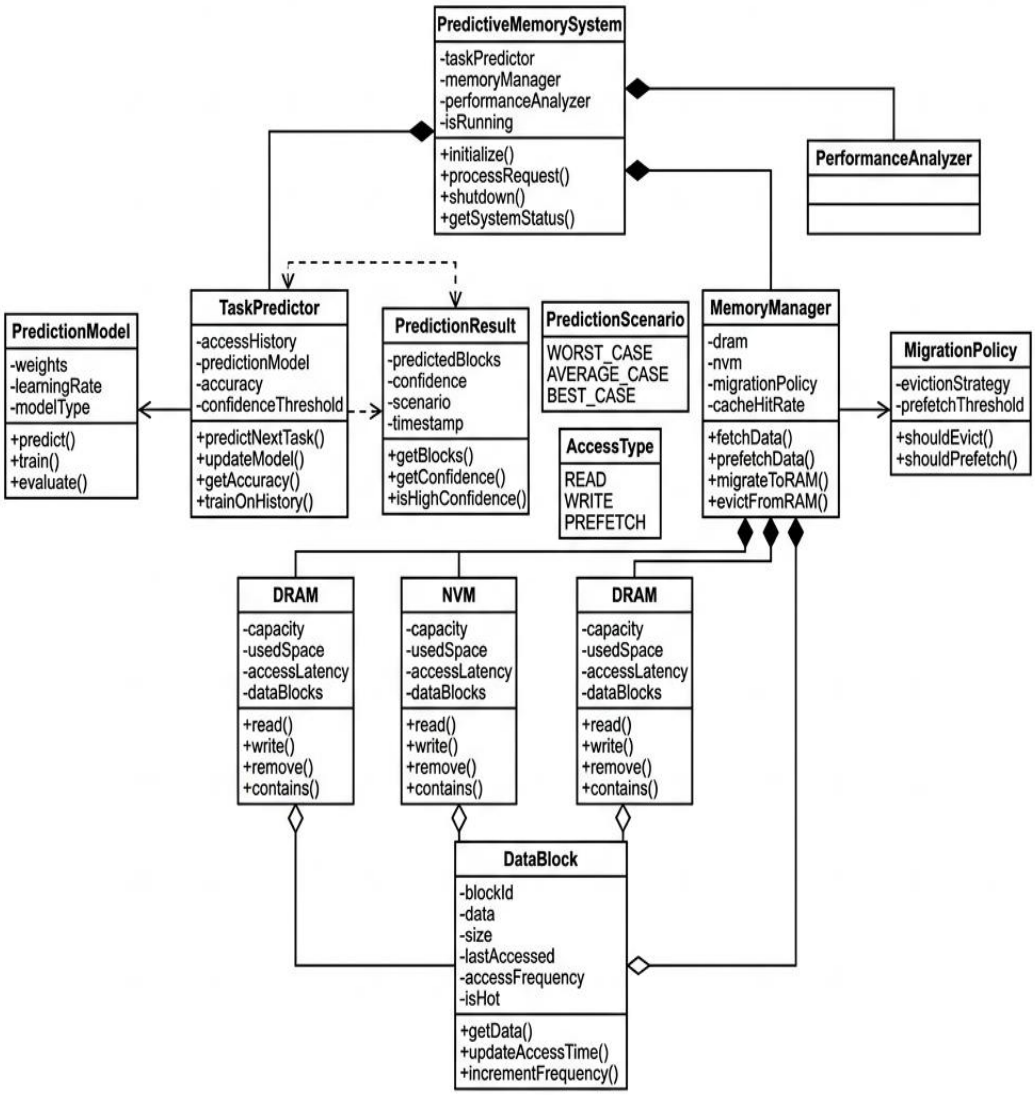
DFD Context Diagram - Predictive Memory Management System



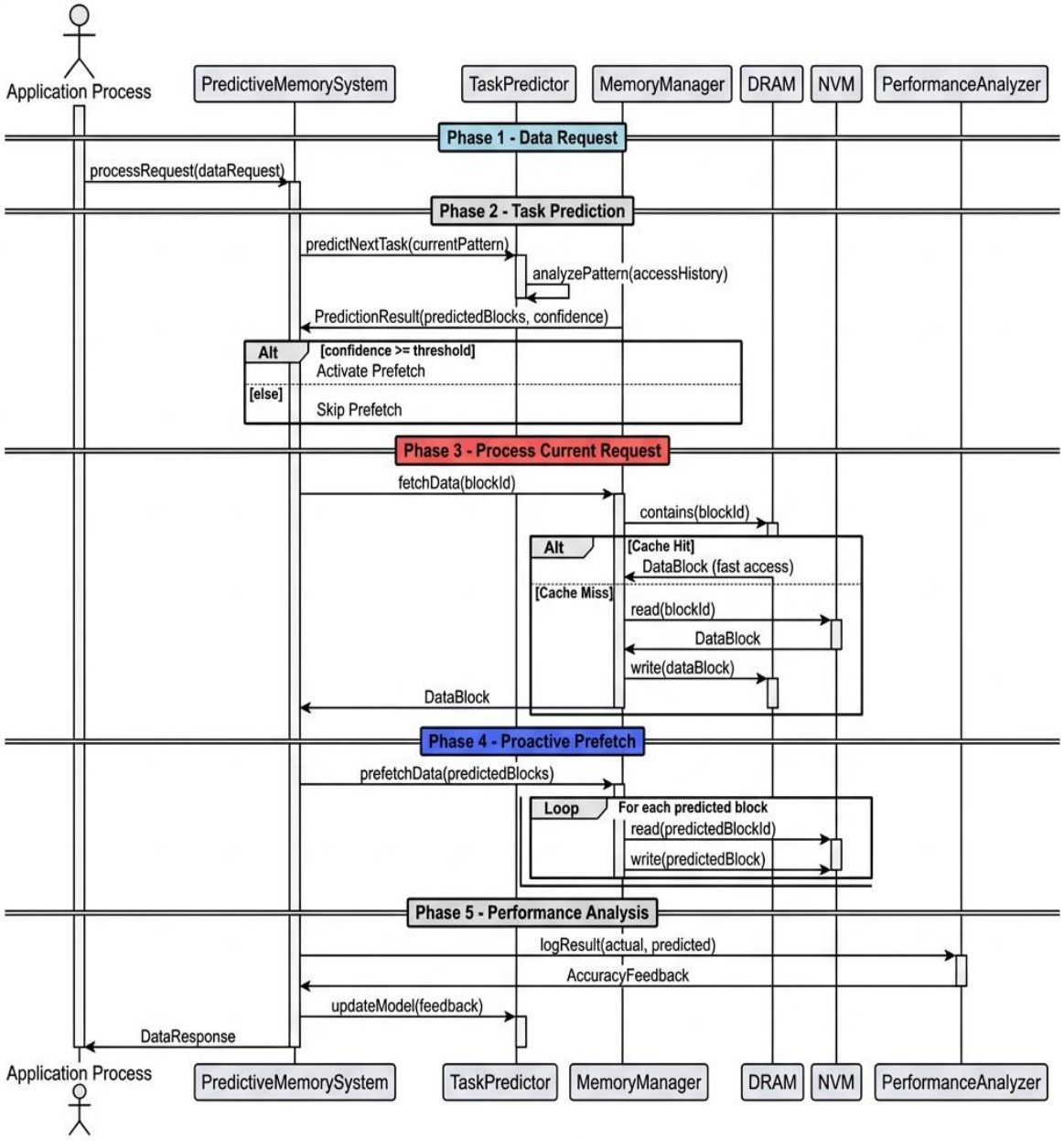
DFD Level 0 - Predictive Memory Management System



UML Class Diagram - Predictive Memory Management System



Sequence Diagram - Proactive Prefetch Flow



Weekly Project Evaluation Rubric

Week	Phase	Deliverables / Tasks	Assessment Criteria	Weight / Score
Week 1-2	**Project Initiation**	Project Proposal & Team Roles	Clarity of objectives, feasibility, and problem definition.	**/10%**
Week 3-4	**Research & Planning**	Literature Review & System Design	Quality of research sources and technical design/logic.	**/15%**
Week 5-7	**Implementation**	Prototype or Initial Code/Draft	Technical execution, progress according to plan, and effort.	**/30%**
Week 8-9	**Testing & Review**	Testing Results & Refinement	Addressing errors/bugs and improving the initial version.	**/15%**
Week 10	**Final Delivery**	Full Report & Final Presentation	Professionalism, clarity, and overall quality of the outcome.	**/30%**